Course Notes RAG and Adaptive Quiz Engine: Grounded Question Answering and Bloom-Calibrated Practice from Lecture PDFs

Royce B Selvaraj
Reg. No. 24BCB0078
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

Srujan Buddharaju
Reg. No. 24BDS0347
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

Pranali Kamisetty
Reg. No. [Registration Number]
School of Computer Science and Engineering
Vellore Institute of Technology
Vellore, India

***Abstract*—Students increasingly ask general-purpose chatbots about their coursework, but the answers are not grounded in the course's own notes and cannot be checked against them. We present a study assistant that answers only from a student's lecture PDFs, cites the exact page of every claim, and generates practice quizzes that adapt to the sections the student knows least. The system parses lecture notes, Beamer slides and scanned handwriting with a layout-aware pipeline that preserves equations as LaTeX, tables and page coordinates; indexes parent–child chunks in a hybrid dense (BGE-M3) and sparse (BM25) index fused by Reciprocal Rank Fusion; reranks parent sections with a cross-encoder and abstains below a relevance threshold; and generates answers with a locally hosted 7-billion-parameter model (Qwen 2.5). A verification layer checks each citation against the retrieved pages, detects citations to the wrong page, highlights the supporting paragraph and completes missing citations. Quiz questions are produced under a JSON schema, calibrated to Bloom's taxonomy, and repaired and validated before display, and a mastery model steers future quizzes toward weak sections. On a ten-question benchmark the system retrieves the correct lecture section for all questions (MRR 1.00); citation coverage of generated answers rose from 29% to 100% of sentences and valid quiz yield from 33% to 100% through the techniques described. The whole system runs offline on one laptop GPU.**

***Index Terms*—retrieval-augmented generation, hybrid retrieval, document layout analysis, citation verification, automatic question generation, adaptive learning, large language models.**

I. Introduction

Large language models (LLMs) have become a default study aid, yet they answer from what they absorbed during training rather than from the material a course actually teaches. Their answers can be fluent and wrong, a failure mode known as hallucination [1], and they rarely point to a page the student can verify. For coursework this is a serious limitation: the lecture notes define the notation, the assumptions and the level of detail that will be examined.

Retrieval-augmented generation (RAG) addresses the problem by retrieving passages from a document collection and conditioning the model on them [2]. Lecture material, however, is hostile to the simple RAG pipelines found in tutorials. Splitting text by character count cuts derivations in half, scrambles the columns of tables, and ignores equations that exist only as images in slides or as handwriting in scanned notes. Pure vector search also blurs exactly the tokens students search for: theorem numbers (“Lemma 3.2”), variable names and Greek letters.

Answering questions is also only half of studying. Practice testing is one of the few learning techniques with consistently high measured utility [3], [4], but writing good multiple-choice questions, with plausible distractors and an explanation for each option, is laborious. A useful assistant should therefore also generate practice questions from the same notes and focus them on what the student has not yet mastered.

This paper describes Course Notes Studio, a system that combines both functions. Its contributions are:

- a layout-aware ingestion pipeline that turns lecture PDFs, slide decks and scanned handwriting into structured blocks carrying LaTeX, tables, page numbers and bounding boxes, and a parent–child chunker that never splits a formula or table;
- a hybrid retrieval funnel (dense + BM25 with a math-aware tokenizer, Reciprocal Rank Fusion, parent promotion and cross-encoder reranking) that abstains with “Not found in course notes” instead of guessing;
- a citation verification layer that checks the source and the page of every citation, highlights the supporting paragraph in the PDF, and completes or flags uncited sentences;
- a structured quiz generator calibrated to Bloom's taxonomy [5], in which the model cites numbered evidence sentences instead of retyping quotes, and drafts are repaired and validated before use;
- an adaptive mastery model that samples weak sections more often, and an evaluation showing which of these techniques made a 7-billion-parameter local model reliable enough for the task.

II. Related Work

A. Retrieval for Question Answering

Dense passage retrieval encodes questions and passages into a shared vector space [6]; BGE-M3 extends this to long, multilingual inputs with a single model [7]. Approximate nearest-neighbour search over such vectors is typically performed with HNSW graphs [8]. Lexical retrieval with BM25 derives from the probabilistic relevance framework [9] and

remains strong on exact-match queries; bm25s provides a fast implementation [10]. Reciprocal Rank Fusion (RRF) combines rankings from different systems without score calibration and often outperforms the individual rankers [11]. Query expansion with hypothetical documents (HyDE) can further improve dense recall [12].

B. Reranking and Grounded Generation

Cross-encoders that read the query and passage together give more accurate relevance estimates than bi-encoders and are widely used as a second stage [13]. RAG systems condition generation on retrieved passages [2], [14], and recent work evaluates whether generated citations actually support the claims they are attached to [15]. Our verification layer is a lightweight, lexical form of this check that runs without an additional model and adds page-level and paragraph-level localization.

C. Document Parsing

Docling combines a page-layout model, the TableFormer table-structure model, OCR and optional formula recognition into a single conversion pipeline [16]. We use it as the primary parser and fall back to PyMuPDF with RapidOCR when it is unavailable or fails on a file.

D. Question Generation and Adaptive Practice

Automatic question generation for education has been surveyed extensively [17]; LLMs make it far easier but raise concerns about correctness and grounding [18]. Bloom's revised taxonomy distinguishes remembering, understanding and applying knowledge [5], which we map to Recall, Conceptual and Application questions. Student modelling ranges from Bayesian knowledge tracing [19] to Elo-style rating updates [20]; our mastery model is an exponential moving average in the spirit of the latter.

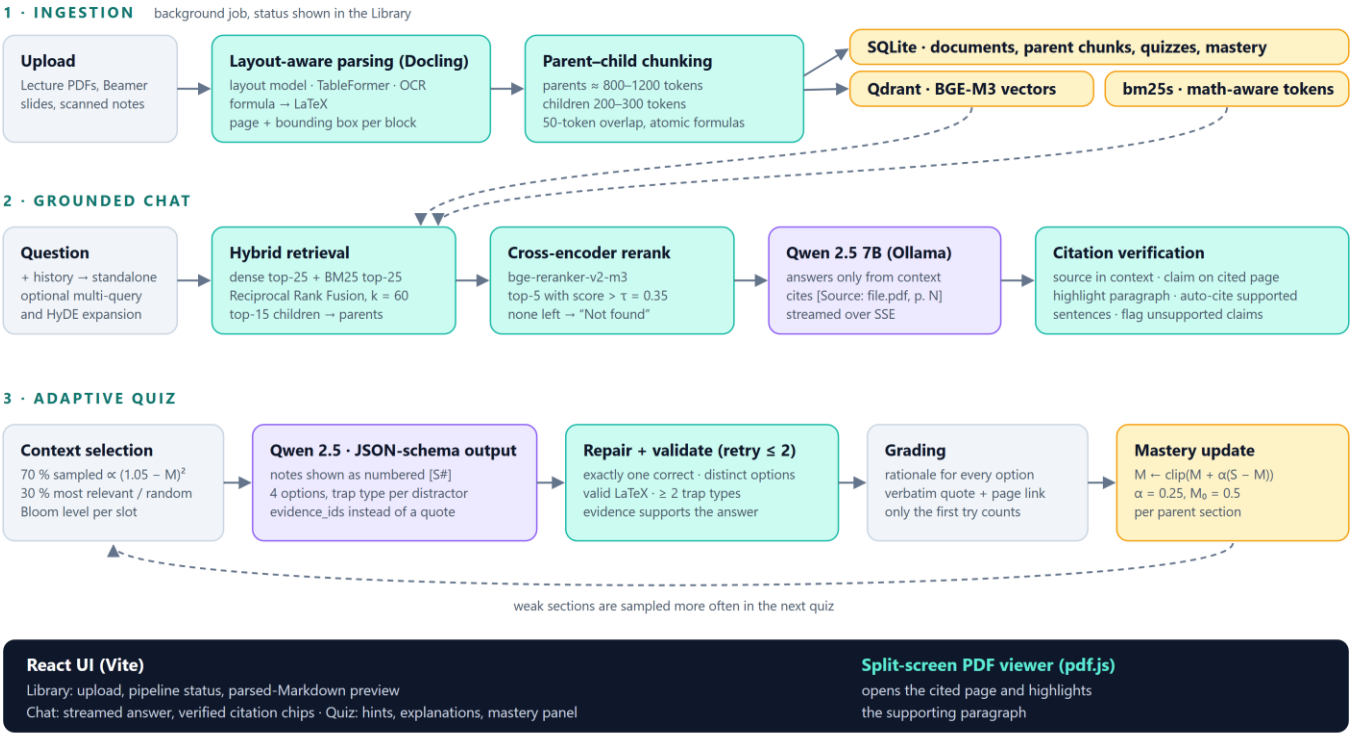


Fig. 1. System architecture. Ingestion (top) runs as a background job; grounded chat (middle) and quiz generation (bottom) run in real time. Dashed arrows show data flowing from the stores and the feedback from mastery to section selection.

III. System Overview

Fig. 1 shows the architecture. It has two workflows. *Ingestion* runs as a background job when a PDF is uploaded: the file is parsed, chunked and indexed, and its status (queued, parsing, chunking, indexing, ready or failed) is visible in the user interface. *Question answering* and *quiz generation* run in real time; answers are streamed to the browser over server-sent events. All components run locally: the language model, Qwen 2.5 7B Instruct [22], is served by Ollama, embeddings and reranking run on the same GPU, and data is stored in SQLite and an embedded Qdrant vector store.

IV. Methodology

A. Layout-Aware Ingestion

Docling's layout model labels each region of a page as a title, section header, paragraph, list item, table, formula or picture. Tables are exported as GitHub-flavoured Markdown, formula regions are converted to LaTeX by a formula-recognition model, and scanned regions are read by OCR. Each resulting block keeps its page number and a bounding box converted to top-left coordinates, which later lets the interface highlight a paragraph in the original PDF. Because the layout model often labels every heading as level 1, heading depth is inferred from

numbering (“2.1”, “Lecture 4”) where available. The fallback parser detects headings from font sizes, recognises formula lines by their density of mathematical symbols and converts Unicode math (Greek letters, super- and subscripts, combining accents) to LaTeX that KaTeX can render; pages with no text layer are passed to RapidOCR.

B. Parent–Child Chunking

Retrieval needs small units for precise matching, while generation needs enough surrounding context to state a derivation completely. We therefore build two levels. *Parent* chunks follow the document structure: a new parent starts at every heading of level 1–3, or at every slide in a slide deck. Small neighbouring sections are merged until a parent reaches about 800 tokens, never beyond 1200 tokens, and never across a heading that begins a new lecture or chapter. *Child* chunks of 200–300 tokens with a 50-token overlap are cut from each parent at sentence boundaries. Formula, table and code blocks are atomic: no chunk boundary may fall inside one. Each child is embedded with a contextual prefix of the form “[Course: CS229-lite | Lecture 4: Support Vector Machines | Section: KKT Conditions]”, and tables are rewritten as one “header: value” sentence per row for embedding, while the Markdown table is kept for display and for the language model.

C. Hybrid Retrieval

A question, rewritten into a standalone question when it follows earlier turns, is searched in two ways. Dense retrieval returns the 25 children with the highest cosine similarity of BGE-M3 embeddings. Sparse retrieval returns the top 25 children under BM25, using a tokenizer that maps Unicode Greek letters and LaTeX commands to the same token (α and `\alpha` both become “alpha”), keeps dotted theorem numbers such as 3.2 and sub- or superscripted variables such as `x_i`, and removes stop words. Optionally, the question is expanded into paraphrases (multi-query) and a hypothetical answer passage (HyDE) [12]. All rankings are fused by RRF [11]:

$$RRF(d) = \sum_{m \in R} \frac{1}{k + r_m(d)}, \; k = 60 \qquad (1)$$

where R is the set of rankings and $r_m(d)$ the 1-based rank of child d in ranking m. The 15 best children are mapped back to their parents (deduplicated), so that the next stages see whole sections.

D. Reranking and Abstention

Each candidate parent is scored together with the question by the bge-reranker-v2-m3 cross-encoder, whose sigmoid output is read as the probability that the passage is relevant. The five highest-scoring parents with a score above $\tau = 0.35$ form the context. If no candidate passes the threshold, the system replies “Not found in course notes” without calling the language model, which removes the most common source of ungrounded answers.

E. Grounded Generation and Citation Verification

The language model receives the parent sections with explicit page markers ([p. 12]) and is instructed to answer only from them, to end every sentence that states a fact with a citation of the form [Source: Lecture_04_SVMs.pdf, p. 2], and to write mathematics in LaTeX. Because a 7B model follows formatting rules more reliably when they are repeated next to the question, the rule is restated at the end of the user turn. After generation, every answer passes four checks. For a claim c and a block b, let T(·) be the set of lexical tokens; the support of b for c is

$$s(c, b) = \frac{|T(c) \cap T(b)|}{|T(c)|} \qquad (2)$$

1) *Source check:* the cited file must be among the retrieved sections and the page inside the section's page range; otherwise the citation is marked unverifiable.
2) *Page check:* if another page of the section supports the claim strongly ($s \geq 0.5$) and clearly better than the cited page (by at least 0.25), the citation is flagged as pointing to the wrong page and linked to the better page.
3) *Highlight:* the block with the highest support on the cited page ($s \geq 0.2$) provides the bounding box that the PDF viewer highlights.
4) *Completion:* sentences without a citation are matched against all retrieved blocks; when one block supports a sentence with $s \geq 0.5$, a citation marked “auto” is inserted, and the remaining sentences are reported as unsupported claims.

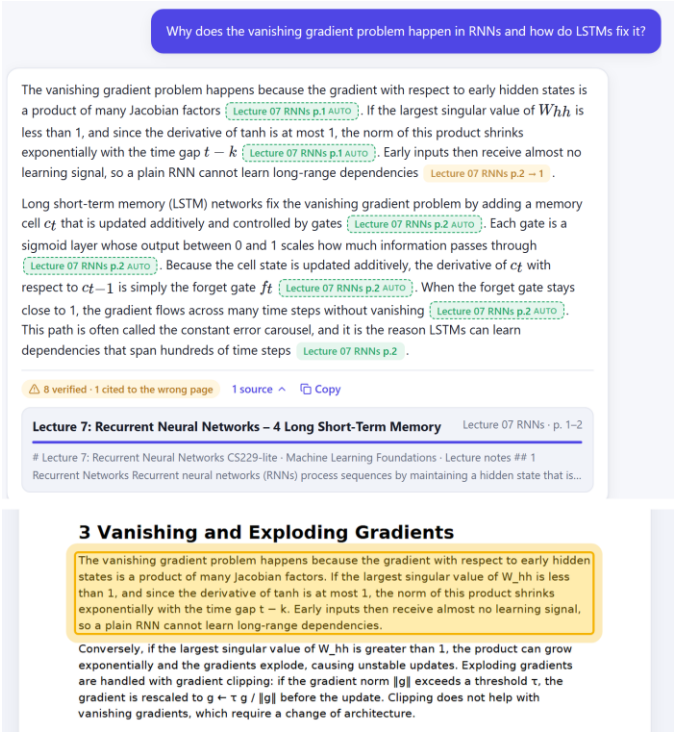


Fig. 2. Grounded chat with the real models (top). Solid green chips are verified citations, dashed “auto” chips were added by completion, and the amber chip “p.2 → 1” was caught citing the wrong page. Clicking a chip opens the PDF with the supporting paragraph highlighted (bottom).

F. Structured Quiz Generation

A quiz request specifies an optional topic, the number of questions (1–10) and the Bloom levels to include. For each question slot one parent section is chosen (Section IV-G) and the model is asked for one question under a JSON schema

enforced by the inference server: a stem, four options each with a correctness flag, a *trap type* and a misconception explanation, a hint, and the evidence for the correct answer. Distractors are built from three pedagogical traps: sign or formula inversion, violation of a theorem's preconditions, and confusion of related terms.

Asking the model to copy an exact quote from the notes proved unreliable: it paraphrased the quote, which then failed verification. Instead, the notes are shown as numbered sentences ([S1], [S2], …) with formulas and tables as single units, and the model returns the numbers of one to three evidence sentences. The quote and its page are assembled from those sentences, so the quote is verbatim by construction. Each draft is then *repaired* and *validated*. Repair fixes labelling slips without changing what the question asks: it labels the single correct option as correct, assigns an unused trap type to a mislabelled distractor, fills an empty distractor rationale from its trap type, and, when the cited evidence does not support the correct option, re-selects the evidence sentence that supports it best if one clears the threshold. Validation then requires exactly one correct option, four distinct options, at least two different trap types, balanced LaTeX, and evidence that shares at least 25% of the correct option's terms. Invalid drafts are sent back with the list of problems, up to two retries, with the sampling temperature raised on each retry (0.4, 0.65, 0.9) so that the model does not repeat the same draft. Options are shuffled before display, and correctness is revealed only after the student answers.

G. Adaptive Mastery

Each parent section c has a mastery score $M_c \in [0, 1]$, initialised to 0.5. After the first attempt at a question drawn from c, with outcome $S = 1$ if correct and 0 otherwise,

$$M_c \leftarrow clip(M_c + \alpha(S - M_c), 0, 1),\ \alpha = 0.25 \qquad (3)$$

Retries are allowed but do not change mastery. When a quiz is generated, 70% of the slots are filled by sampling sections without replacement with probability

$$P(c) \propto (1.05 - M_c)^2 \qquad (4)$$

so that a section at 10% mastery is about 40 times as likely to be chosen as one at 90%. The remaining 30% are the most relevant sections for a topic quiz, or uniformly random sections otherwise, which keeps coverage of the whole course. Bloom levels are assigned to slots in rotation.

V. Implementation

The system is implemented in about 5,700 lines of Python and 3,700 lines of TypeScript, with 3,000 lines of tests. Table I lists the main components. The backend exposes a REST interface for courses, documents (upload, status, parsed Markdown, file, re-processing and deletion), retrieval, chat, quizzes, answers and mastery; chat answers and quiz questions are streamed as server-sent events so that the interface can show progress. All thresholds and model names are configuration settings that can be overridden by environment variables. A *test profile* replaces the neural models with deterministic stand-ins (a hashing embedder, a lexical reranker and an extractive answer generator), so that the complete product can be exercised end to end without a GPU.

The interface (Fig. 2) has three tabs. The *Library* accepts PDFs by drag and drop, shows each document's progress through the pipeline and previews the extracted Markdown with rendered mathematics. *Chat* streams answers with citation chips coloured by their verification status; clicking a chip opens a split-screen PDF viewer on the cited page with the supporting paragraph highlighted, and an unverifiable citation is struck through. *Quiz* generates questions progressively, accepts answers by mouse or keyboard, explains every option with its trap type, links the source quote to its page, and shows a per-section mastery panel from which a weak section can be practised directly.

TABLE I
Technology Stack

Component	Implementation
Parsing	Docling 2.143 (layout, TableFormer, formula model); PyMuPDF 2.0 + RapidOCR fallback
Embeddings	BAAI/bge-m3 (1024-d, FP16 on GPU) in an embedded Qdrant collection
Sparse index	bm25s with a math-aware tokenizer, rebuilt per course from SQLite
Reranker	BAAI/bge-reranker-v2-m3 cross-encoder, max length 1024
LLM	Qwen 2.5 7B Instruct (Q4) via Ollama; any OpenAI-compatible server
Backend	Python 3.12, FastAPI, SQLite, server-sent events
Frontend	React 19, Vite, react-pdf (pdf.js), KaTeX, react-markdown
Testing	pytest, Vitest + Testing Library, Playwright

VI. Evaluation

A. Setup

All measurements were taken on a laptop with an Intel Core i9-14900HX, 32 GB of RAM and an NVIDIA GeForce RTX 4070 Laptop GPU with 8 GB of memory. The test corpus is a synthetic course of four PDFs (13 pages): lecture notes on support vector machines and on recurrent neural networks containing equations rendered as text and as images and two tables, a seven-slide Beamer-style probability deck with an image-only equation, and a scanned page of handwritten notes on gradient descent. Ingestion produced 12 parent sections and 23 child chunks. The retrieval benchmark consists of ten hand-written questions that require paraphrase matching, exact theorem numbers, a table lookup and OCR. For each question we record the rank of the first retrieved section from the expected document (Hit@1, Hit@5, mean reciprocal rank) and whether the expected page lies inside that section (page accuracy).

B. Retrieval

Table II lists the per-question results with the real models. In the first run the only miss was the question about the variance of a Bernoulli variable, whose answer is a row of a table; no section passed the reranking threshold, so the system correctly

abstained rather than answering from the wrong section. Rewriting table rows as sentences for embedding and reranking (Section IV-B) lifted that question above the threshold (0.38 > 0.35) and also raised the score of the kernel-table question from 0.61 to 0.86. Table III summarises the three configurations. The offline stand-ins test the pipeline rather than the models and are reported for comparison only. With warm models a query takes 268 ms on average; the first query of a session takes about 10.7 s because the models are loaded onto the GPU.

TABLE II
PER-QUESTION RETRIEVAL (RANK AND RERANKER SCORE)

#	Question	Doc	First run	Final
1	Vanishing gradient & LSTM fix	RNN	1 (1.00)	1 (1.00)
2	KKT & complementary slackness	SVM	1 (0.99)	1 (0.98)
3	What Lemma 3.2 states	SVM	1 (0.67)	1 (0.66)
4	Bayes' theorem	Slides	1 (0.98)	1 (0.98)
5	Zero covariance vs. independence	Slides	1 (1.00)	1 (1.00)
6	Geometric vs. functional margin	SVM	1 (0.95)	1 (0.92)
7	Infinite-dimensional kernel	SVM	1 (0.61)	1 (0.86)
8	Gradient clipping	RNN	1 (0.96)	1 (0.96)
9	Bernoulli variance (table)	Slides	not found	1 (0.38)
10	Large learning rate (scan)	Scan	1 (1.00)	1 (1.00)

TABLE III
RETRIEVAL BENCHMARK SUMMARY

Configuration	Hit@1	Hit@5	MRR	Page acc.
Offline stand-ins	1.00	1.00	1.00	0.90
Real models, first run	0.90	0.90	0.90	0.90
Real models, final	1.00	1.00	1.00	1.00

C. Grounding of Generated Answers

For the question "Why does the vanishing gradient problem happen in RNNs and how do LSTMs fix it?", Qwen 2.5 7B initially placed a single citation at the end of each paragraph: 2 citations, both valid, but only 29% of factual sentences carried one. Restating the citation rule in the user turn produced 6 valid citations, no invalid ones, and citations on 100% of sentences; the page check flagged one of them, which cited page 1 for a passage on page 2. The model's behaviour varies between runs, which is why completion matters: in the run shown in Fig. 2 it cited only 2 of 8 sentences, one of them to the wrong page, and completion supplied the other 6, so every factual sentence still ended with a verified citation (8 verified, 1 corrected page).

D. Quiz Generation Reliability

Table IV shows the fraction of requested questions that passed validation for a three-question quiz on KKT conditions and support vectors. With free-text quotes, two of three slots failed because the model paraphrased the quote. Numbered evidence sentences raised the yield to 67% over four runs; the remaining failures were labelling slips that the model repeated on every retry, such as a correct option without the "correct" trap label, an empty rationale or evidence about a neighbouring fact. With repair and a rising retry temperature, 9 of 9 questions passed across three seeds, with 2 evidence re-selections and 2 filled rationales; generation took about 36 s per question.

TABLE IV
VALID QUIZ QUESTIONS PER REQUEST

Configuration	Valid / requested
Free-text exact quote	1 / 3 (33%)
Numbered evidence sentences	8 / 12 (67%)
+ repair and rising retry temperature	9 / 9 (100%)

E. Ingestion Performance and Software Testing

Ingesting the four documents with Docling, embedding and indexing took 41 s on the GPU. Docling recovered the image-only Bayes equation on the slides as LaTeX (`P(A|B)=\frac{P(B|A)P(A)}{P(B)}`), kept both tables, detected the slide deck and read the handwritten page by OCR; it misread one symbol ($\hat{\gamma}$ as $\hat{Y}$) in a text equation. The software is covered by 289 backend unit and API end-to-end tests, 56 frontend unit tests, 9 browser end-to-end tests on desktop and mobile viewports, and 5 live tests that run the complete stack with the real models. Every component is tested with positive and negative cases, including empty, non-PDF, encrypted, truncated and duplicate uploads, off-topic questions, and malformed model output.

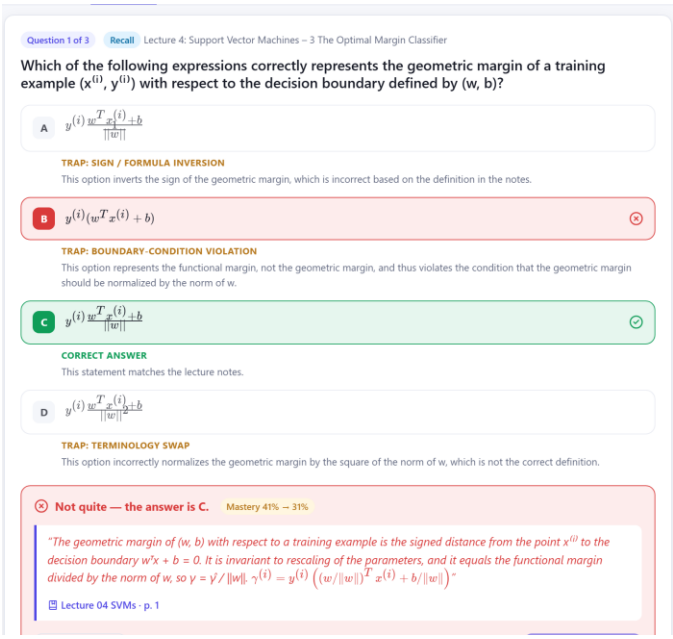


Fig. 3. Feedback after answering a generated question: every option shows its trap type and rationale, the verbatim source quote links to its page, and the change in mastery is shown.

VII. RELATION TO THE BCSE306L SYLLABUS

The project was developed for BCSE306L Artificial Intelligence, whose textbook is Russell and Norvig [21]. Table V maps each syllabus module to the part of the system that applies it. Module 7 (communicating, perceiving and acting) is

the core of the project: information retrieval, information extraction and probabilistic language processing are its main subsystems, and layout detection and OCR are perception tasks on page images. Module 5 appears in the probabilistic ranking models, the relevance threshold that turns uncertainty into an explicit abstention, and the mastery score as a belief about the student. Searching (Module 2) underlies retrieval, and rule-based reasoning (Module 4) underlies both verification layers. Local search and planning are used in a looser sense, in the generate–repair–retry loop and the hierarchical assembly of a quiz.

TABLE V
MAPPING TO SYLLABUS MODULES

Module	Where it appears	Link
1. Introduction, Intelligent Agents	Learning agent: perceives notes and questions, acts by answering and quizzing, keeps a student model; partially observable, sequential environment.	Strong
2. Search	Retrieval as search over passages: HNSW best-first graph search, BM25 ranking, RRF fusion, top-k pruning.	Strong
3. Local and Adversarial Search	Generate-and-test quiz drafting with repair and restarts at rising temperature; distractors as the most tempting wrong answers.	Partial
4. Logic and Reasoning	Rule-based verification of citations and quiz drafts; "exactly one correct option" as a propositional constraint.	Strong
5. Uncertain Knowledge	Probabilistic relevance (BM25), reranker probability and abstention threshold, mastery as an updated belief.	Strong
6. Planning	Hierarchical quiz assembly (goal → slots → section → level → generate → validate); tracked ingestion plan with fallback.	Partial
7. Communicating, Perceiving, Acting	Information retrieval, information extraction (layout, tables, formulas), probabilistic language processing, OCR and layout detection.	Strong
8. Contemporary Issues	Hallucination and grounding, verifiable citations, private on-device inference.	Strong

The project also addresses the four course outcomes: evaluating AI methods (CO1) through the retrieval benchmark and the before/after measurements of each technique; applying AI to inference, perception, knowledge representation and learning (CO2); reasoning under uncertainty with a knowledge representation of parent and child chunks (CO3); and showing how search drives problem solving (CO4), since retrieval decides what the language model is allowed to say.

VIII. DISCUSSION AND LIMITATIONS

The measurements show that a small local model becomes usable for grounded study support when the surrounding system carries most of the burden: retrieval decides what may be said, the abstention threshold decides when nothing should be said, and verification and repair catch the formatting slips that a 7B model makes. Several limitations remain. The evaluation is small and uses synthetic lecture notes written for the purpose; the results need confirming on real course material with independently written questions and with student and faculty ratings of quiz quality. The verification layer is lexical: it can confirm that a claim's terms appear on a page, but not that a paraphrased claim is entailed by it. Generation remains stochastic, so a quiz slot can still be skipped after all retries, and an answer can omit part of a question. Distinct options are checked on their text, so two options can still look almost the same once rendered (options A and C in Fig. 3). Formula recognition can misread symbols, and parsing is the slowest step when no GPU is available.

IX. CONCLUSION AND FUTURE WORK

We presented a study assistant that answers questions from a student's own lecture PDFs with verified page-level citations and generates adaptive, Bloom-calibrated quizzes from the same material, entirely on one laptop. Layout-aware parsing, hybrid retrieval with reranking and abstention, citation verification and completion, numbered evidence for quiz generation, and draft repair together raised retrieval accuracy to 10 of 10 questions, citation coverage to 100% of sentences and quiz yield to 100% in our tests. Future work includes a larger evaluation on real courses, an entailment model in the verification layer, and replacing the moving-average mastery score with Bayesian knowledge tracing [19] and spaced-repetition scheduling.

REFERENCES

[1] Z. Ji *et al.*, "Survey of hallucination in natural language generation," *ACM Comput. Surv.*, vol. 55, no. 12, pp. 1–38, 2023.

[2] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst. (NeurIPS)*, 2020, pp. 9459–9474.

[3] J. Dunlosky, K. A. Rawson, E. J. Marsh, M. J. Nathan, and D. T. Willingham, "Improving students' learning with effective learning techniques," *Psychol. Sci. Public Interest*, vol. 14, no. 1, pp. 4–58, 2013.

[4] H. L. Roediger and J. D. Karpicke, "Test-enhanced learning: Taking memory tests improves long-term retention," *Psychol. Sci.*, vol. 17, no. 3, pp. 249–255, 2006.

[5] L. W. Anderson and D. R. Krathwohl, Eds., *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Bloom's Taxonomy of Educational Objectives*. New York, NY, USA: Longman, 2001.

[6] V. Karpukhin *et al.*, "Dense passage retrieval for open-domain question answering," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2020, pp. 6769–6781.

[7] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "BGE M3-Embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," 2024, *arXiv:2402.03216*.

[8] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020.

[9] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.

[10] X. H. Lù, "BM25S: Orders of magnitude faster lexical search via eager sparse scoring," 2024, *arXiv:2407.03618*.

[11] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf. Res. Develop. Inf. Retr.*, 2009, pp. 758–759.

[12] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise zero-shot dense retrieval without relevance labels," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2023, pp. 1762–1777.

[13] R. Nogueira and K. Cho, “Passage re-ranking with BERT,” 2019, *arXiv:1901.04085*.

[14] Y. Gao *et al.*, “Retrieval-augmented generation for large language models: A survey,” 2023, *arXiv:2312.10997*.

[15] T. Gao, H. Yen, J. Yu, and D. Chen, “Enabling large language models to generate text with citations,” in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2023, pp. 6465–6488.

[16] C. Auer *et al.*, “Docling technical report,” 2024, *arXiv:2408.09869*.

[17] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, “A systematic review of automatic question generation for educational purposes,” *Int. J. Artif. Intell. Educ.*, vol. 30, pp. 121–204, 2020.

[18] E. Kasneci *et al.*, “ChatGPT for good? On opportunities and challenges of large language models for education,” *Learn. Individual Differences*, vol. 103, 2023, Art. no. 102274.

[19] A. T. Corbett and J. R. Anderson, “Knowledge tracing: Modeling the acquisition of procedural knowledge,” *User Model. User-Adapted Interact.*, vol. 4, no. 4, pp. 253–278, 1994.

[20] R. Pelánek, “Applications of the Elo rating system in adaptive educational systems,” *Comput. Educ.*, vol. 98, pp. 169–179, 2016.

[21] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2015.

[22] Qwen Team, “Qwen2.5 technical report,” 2024, *arXiv:2412.15115*.